

Student 25 **PATAN MOHAMMED RIYAZ KHAN** 192572126RC

20 / 20 submitted

Q1. Void Function (No Return Value)

CODE

```
def display():  
    print("Welcome to Python Programming")  
display()
```

OUTPUT (no output)

Q2. Void Function (No Return Value)

CODE

```
def mul(n):  
    for i in range(1,11):  
        print(n,"X",i,"=", (n*i))  
n=int(input())  
mul(n)
```

INPUT 5

OUTPUT (no output)

Q3. Void Function (No Return Value)

CODE

```
def num(n):  
    for i in range(1,n+1):  
        if n%i==0:  
            print(i)  
a=int(input())  
num(a)
```

INPUT 10

OUTPUT (no output)

Q4. Fruitful Function (Returns One Value)

CODE

```
def string(text):  
    vowels='AEIOUaeiou'  
    count=0  
    for ch in text:  
        if ch in vowels:  
            count+=1  
    return count  
n=input()  
print("Vowels=",string(n))
```

INPUT riyaz

OUTPUT (no output)

Q5. Fruitful Function (Returns One Value)

CODE

```
def find_largest(lst):  
    return max(lst)  
  
numbers = list(map(int, input().split()))  
print(find_largest(numbers))
```

INPUT

1 2 3 4 5

OUTPUT

(no output)

Q6. Fruitful Function (Returns One Value)

CODE

```
def is_prime(n):  
    if n < 2:  
        return False  
    for i in range(2, int(n**0.5) + 1):  
        if n % i == 0:  
            return False  
    return True  
  
num = int(input("Enter a number: "))  
result = is_prime(num)  
if result:  
    print(f"{num} is a Prime number")  
else:  
    print(f"{num} is not a Prime number")
```

INPUT

7

OUTPUT

(no output)

Q7. Function Returning Two or More Values

CODE

```
def student_result(m1, m2, m3):
    total = m1 + m2 + m3
    average = total / 3

    if average ≥ 90:
        grade = "A+"
    elif average ≥ 80:
        grade = "A"
    elif average ≥ 70:
        grade = "B"
    elif average ≥ 60:
        grade = "C"
    elif average ≥ 50:
        grade = "D"
    else:
        grade = "F"

    return total, average, grade

m1, m2, m3 = map(float, input("Enter marks for Subject 1, 2, and 3: ").split())

total, average, grade = student_result(m1, m2, m3)

print("Total:", total)
print("Average:", average)
print("Grade:", grade)
```

INPUT 90 92 94

OUTPUT (no output)

Q8. Function Returning Two or More Values

CODE

```
import math

def circle_area_circumference(radius):
    area = math.pi * radius * radius
    circumference = 2 * math.pi * radius
    return area, circumference

r = float(input("Enter radius: "))
a, c = circle_area_circumference(r)
print("Area:", a)
print("Circumference:", c)
```

INPUT 10

OUTPUT (no output)

Q9. Function Returning Two or More Values

CODE

```
import math

def square_cube_sqrt(n):
    square = n ** 2
    cube = n ** 3
    sqrt = math.sqrt(n) # works for n ≥ 0
    return square, cube, sqrt

# Example
x = float(input("Enter a number: "))
sq, cu, rt = square_cube_sqrt(x)
print("Square:", sq)
print("Cube:", cu)
print("Square Root:", rt)
```

INPUT 4

OUTPUT (no output)

Q10. Keyword Arguments

CODE

```
def student(name, age, department):
    print(f"Name: {name}")
    print(f"Age: {age}")
    print(f"Department: {department}")
student(name="RIYAZ KHAN", age=19, department="CSE - AI")
```

OUTPUT (no output)

Q11. Keyword Arguments

CODE

```
def employee(empid, name, salary):
    print("Employee ID:", empid)
    print("Name:", name)
    print("Salary:", salary)

# Calling by changing the order using keywords
employee(name="Riyaz Khan", salary=50000, empid=192572126)
```

OUTPUT (no output)

Q12. Keyword Arguments

CODE

```
def book(title, author, price):
    print(f"Title: {title}")
    print(f"Author: {author}")
    print(f"Price: {price}")

book(title="The Alchemist", author="Paulo Coelho", price=499)
```

OUTPUT (no output)

Q13. Variable-Length Arguments (*args)

CODE

```
def sum_numbers(*args):  
    return sum(args)  
print(sum_numbers(1, 2, 3, 4))
```

OUTPUT (no output)

Q14. Variable-Length Arguments (*args)

CODE

```
def average_marks(*marks):  
    if len(marks) == 0:  
        return 0  
    return sum(marks) / len(marks)  
  
print(average_marks(80, 90, 70, 85))
```

OUTPUT (no output)

Q15. Variable-Length Arguments (*args)

CODE

```
def print_strings(*args):  
    for s in args:  
        print(s)  
  
print_strings("Hello", "World", "Python")
```

OUTPUT (no output)

Q16. Variable-Length Arguments (*args)

CODE

```
def largest(*args):  
    if len(args) == 0:  
        return None  
    return max(args)  
  
print(largest(10, 5, 20, 3))  
print(largest(1))
```

OUTPUT (no output)

Q17. Recursive Functions

CODE

```
def fibonacci(n):
    def fib_recursive(k):
        if k == 0:
            return 0
        if k == 1:
            return 1
        return fib_recursive(k - 1) + fib_recursive(k - 2)

    series = []
    for i in range(n):
        series.append(fib_recursive(i))
    return series

n = int(input("Enter number of terms: "))
print(*fibonacci(n))
```

INPUT

2

OUTPUT

(no output)

Q18. Recursive Functions

CODE

```
def add_all(*args):
    total = 0
    for x in args:
        total += x
    return total

print(add_all(1, 2, 3, 4))
```

OUTPUT

(no output)

Q19. Recursive Functions

CODE

```
def reverse_string(s):
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]

text = "SIMATS"
print("Reversed string:", reverse_string(text))
```

OUTPUT

(no output)

Q20. Recursive Functions

CODE

```
def power(x, n):  
    if n == 0:  
        return 1  
    if n < 0:  
        return 1 / power(x, -n)  
    return x * power(x, n - 1)  
  
inp = input("Enter x and n (e.g., 10.0 2): ").split()  
x = float(inp[0])  
n = int(inp[1])  
  
print("Result:", power(x, n))
```

INPUT

10.0 2

OUTPUT

(no output)